FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Eric V. Tividad Jr.
Section:	AN 42
Professor:	Ms. Crisola Tan
Date:	10/ /2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
    - I think it'll be the Rule-Based Reasoning. Compared to the CBR, it's much more versatile to implement due to it being mostly using of IF's statements than on CBR, which needs to have a similarity check and the proper case attributes. Though I'd say that CBR is probably much more reliable in terms of producing reliable outputs as it is gone through a process than on RBR.

2. Which is more flexible in solving unstructured problems?
    - I think it'll be CBR for this one. This is mainly because an unstructured problem attribute can't be defined clearly, which the RBR strength point. Instead of basing it from attributes and making rules around it, CBR finds the most similar case for it then allocate those attributes to the said unstructured problem and can even adjust it to its liking.

3. Which is better for situations with continuously changing data?
    - CBR will be great for this situation. One of the limitations that I see in RBR systems is that they are quite reliant on already declared data. CBR systems can overpass it as it doesn't

rely on it totally. Even if there is a new data inputted, its similarity will still be determined regardless and do its functionality well.

4. Examples of real-world applications for both.
   - For the CBR algorithm, I think it'll be great for "planning" systems; systems where it helps users gain useful insights for such situation. Instead of preparing for what we're expected to arrive, we prepare from look back at history. Though simple, these insights are still helpful as their chances that we may overlook something. Examples of such system are medical diagnosis, product advisors, or even engineering tools comparison systems. On the other hand, RBR systems are great algorithms to serve as "guards" on such processes, as they can lock out anything that doesn't goes along with their criteria. For example, filtering, sorting, and even traffic light violator detector are great applications for this algorithm.

Your Code - Laboratory Tasks ⇩

```
# RULE-BASED REASONING SECTION
# The knowledge base: "player" stats
facts = {
    "status": {
        "health": 70, # out of 100
        "hunger": 75, # out of 100, higher = hungrier
        "is_bitten": False,
    },
    "supplies": {
        "ammo": 3,
        "water_bottles": 0,
        "has_medkit": True,
    },
    "surroundings": {
        "zombies_nearby": 12,
        "is_night": True,
        "in_shelter": False,
        "has_vehicle": False,
    },
}

# low health & has medkit = heal bradar
def rule_low_health(facts):
    if facts["status"]["health"] < 50 and facts["supplies"]["has_medkit"]:
        return "Use the medkit to heal"

# hungry = look for food pls
def rule_hungry(facts):
```

```
    if facts["status"]["hunger"] > 70:
        return "Scavenge for food"

# no water = look for water source
def rule_no_water(facts):
    if facts["supplies"]["water_bottles"] == 0:
        return "Find a clean water source"

# surrounded by zombies & low ammo = don't fight
def rule_surrounded(facts):
    if facts["surroundings"]["zombies_nearby"] > 10 and facts["supplies"]["ammo"] < 5:
        return "Avoid combat and run or hide"

# night time and not in shelter = look for shelter
def rule_night(facts):
    if facts["surroundings"]["is_night"] and not facts["surroundings"]["in_shelter"]:
        return "Find a shelter first"

# bitten = gtfo of the group
def rule_bitten(facts):
    if facts["status"]["is_bitten"]:
        return "Isolate yourself from the group"

# surrounding has vehicles and not many zombie is arround = USE THE VEHICLE BRADAR
def rule_vehicle(facts):
    if facts["surroundings"]["has_vehicle"] and facts["surroundings"]["zombies_nearby"] > 0:
        return "Escape using the vehicle"

# list of all the rules
rules = [rule_low_health, rule_hungry, rule_no_water, rule_surrounded,
         rule_night, rule_bitten, rule_vehicle]

# the inference machine!
# runs all rules function then stores it in a array
def inference_engine(facts, rules):
    actions = []
    for rule in rules:
        result = rule(facts)
        if result is not None:
            actions.append(result)
    return actions

# CASE-BASED REASONING SECTION
# (the case base is made before the loop so saved cases are kept every round)
```

```
case_base = [
    {"problem": {"zombies_nearby": "many",
                 "ammo": "low",
                 "in_shelter": False,
                 "is_night": True
                 },
     "solution": "Sneak away quietly and hide in a building until sunrise."
     },

    {"problem": {"zombies_nearby": "few",
                 "ammo": "high",
                 "in_shelter": False,
                 "is_night": False
                 },
     "solution": "Clear the zombies with controlled shots and loot the area."
     },

    {"problem": {"zombies_nearby": "many",
                 "ammo": "high",
                 "in_shelter": True,
                 "is_night": True
                 },
     "solution": "Barricade the doors and shoot from the windows."
     },

    {"problem": {"zombies_nearby": "few",
                 "ammo": "low",
                 "in_shelter": True,
                 "is_night": False
                 },
     "solution": "Stay put, rest, and craft melee weapons."
     },
]

# similarity check
def similarity(new_problem, old_problem):
    matches = 0
    for feature in new_problem:
        if new_problem[feature] == old_problem[feature]:
            matches = matches + 1
    return matches / len(new_problem)

# MAIN LOOP: runs RBR, then CBR, then asks if the user wants to go again
while True:
```

```
print("Zombie Apocalypse Survival Guide")
print("PLAYER STATS:")
for category in facts:
    print("[" + category + "]")
    for name in facts[category]:
        print("  " + name + ": " + str(facts[category][name]))
print()

# just prints out the possible actions based from the rules and facts we have
actions = inference_engine(facts, rules)
print("Rule-Based Reasoning Actions:")
if len(actions) == 0:
    print("- No rules fired")
for action in actions:
    print("- " + action)
print()

print("Describe your situation:")
zombies = input("How many zombies are nearby? (few/many): ").strip().lower()
ammo = input("How is your ammo? (low/high): ").strip().lower()
shelter = input("Are you in a shelter? (yes/no): ").strip().lower()
night = input("Is it night? (yes/no): ").strip().lower()

new_problem = {
    "zombies_nearby": zombies,
    "ammo": ammo,
    "in_shelter": shelter == "yes",
    "is_night": night == "yes",
}

# the 4R's of CBR!
# RETRIEVE: Finds the most similar case for the new case
print("")
print("Case-Based Reasoning Solution")
print("Checking similarity with other cases:")
best_case = None
best_score = -1
number = 1
for case in case_base:
    score = similarity(new_problem, case["problem"])
    print("Similarity with case " + str(number) + ": " + str(round(score, 2)))
    if score > best_score:
        best_score = score
        best_case = case
```

```
    number = number + 1

# REUSE: use the solution of the best case
solution = best_case["solution"]
print("Proposed solution: " + solution)

# REVISE: the user can change the solution
print("")
answer = input("Would you like to change the proposed solution? (y/n): ").strip().lower()

if answer == "y":
    new_solution = input("Enter new solution (or Enter to keep it): ").strip()
    if new_solution != "":
        solution = new_solution
        print("Solution changed")
    else:
        print("Nothing entered, solution kept")
else:
    print("Solution accepted")

# RETAIN: save the new case so it can be used next time
case_base.append({"problem": new_problem, "solution": solution})
print("Case saved. The case base now has " + str(len(case_base)) + " cases.")
print("\nFinal solution: " + solution)

# ask if the user wants to run again or quit
print("")
again = input("Run again or quit? (y/n): ").strip().lower()
while again != "y" and again != "n":
    again = input("Please type y to run again or n to quit: ").strip().lower()
if again == "n":
    print("Stay alive out there!")
    break
print("------------------------------")
```

Output – Main program flow:

```
Zombie Apocalypse Survival Guide
PLAYER STATS:
[status]
  health: 70
  hunger: 75
  is_bitten: False
[supplies]
  ammo: 3
  water_bottles: 0
  has_medkit: True
[surroundings]
  zombies_nearby: 12
  is_night: True
  in_shelter: False
  has_vehicle: False

Rule-Based Reasoning Actions:
- Scavenge for food
- Find a clean water source
- Avoid combat and run or hide
- Find a shelter first

Describe your situation:
How many zombies are nearby? (few/many): many
How is your ammo? (low/high): low
Are you in a shelter? (yes/no): no
Is it night? (yes/no): yes

Case-Based Reasoning Solution
Checking similarity with other cases:
Similarity with case 1: 1.0
Similarity with case 2: 0.25
Similarity with case 3: 0.5
Similarity with case 4: 0.25
Proposed solution: Sneak away quietly and hide in a building until sunrise.
```

```
Would you like to change the proposed solution? (y/n): y
Enter new solution (or Enter to keep it): sneak away, find supplies, and find a building to hide until sunrise
Solution changed
Case saved. The case base now has 5 cases.

Final solution: sneak away, find supplies, and find a building to hide until sunrise

Run again or quit? (y/n): y
------------------------------
Zombie Apocalypse Survival Guide
PLAYER STATS:
[status]
  health: 70
  hunger: 75
  is_bitten: False
[supplies]
  ammo: 3
```

New inputs with ending program input:

```
Describe your situation:
How many zombies are nearby? (few/many): few
How is your ammo? (low/high): high
Are you in a shelter? (yes/no): yes
Is it night? (yes/no): no

Case-Based Reasoning Solution
Checking similarity with other cases:
Similarity with case 1: 0.0
Similarity with case 2: 0.75
Similarity with case 3: 0.5
Similarity with case 4: 0.75
Similarity with case 5: 0.0
Proposed solution: Clear the zombies with controlled shots and loot the area.

Would you like to change the proposed solution? (y/n): n
Solution accepted
Case saved. The case base now has 6 cases.

Final solution: Clear the zombies with controlled shots and loot the area.

Run again or quit? (y/n): n
Stay alive out there!
```